How Do Humans Write Code? Large Models Do It the Same Way Too

Anonymous ACL submission

Abstract

Program-of-Thought (PoT) replaces natural language-based Chain-of-Thought (CoT) as the most popular method in Large Language Models (LLMs) mathematical reasoning tasks by utilizing external tool calls to circumvent computational errors. However, our evaluation of the GPT-4 and Llama series reveals that using PoT introduces more reasoning errors, such as incorrect formulas or flawed logic, compared to CoT. To address this issue, we propose Human-Think Language (HTL), which leverages a suite of strategies that help integrate PoT and CoT, encompassing: (1) a new generation paradigm that uses full CoT reasoning to control code generation. (2) Focus Attention, that directs model attention to the CoT reasoning during PoT to generate more logical code. (3) reinforcement learning that utilizes the accuracy of both CoT and PoT responses as rewards to prevent repetitive reasoning steps in LLMs when solving difficult math problems. Our method achieves an average improvement of 6.5% on the Llama-Base model and 4.3% on the Mistral-Base model across 8 mathematical calculation datasets. It also shows significant effectiveness on five out-of-domain datasets by controlling the model's information flow, exhibiting strong transferability. Additionally, HTL shows the most significant improvement in non-mathematical natural language inference task, contributing to a unified reasoning task framework.

1 Introduction

Solving Mathematical reasoning problems is a significant challenge for current LLMs (Madaan et al., 2022; OpenAI et al., 2023). This task requires interpreting information, identifying relevant mathematical concepts, and formulating equations to solve the problems (Ahn et al., 2024). Due to computational errors in LLMs (Wei et al., 2023; Gao et al., 2023), using CoT (Wang et al., 2023b; Wei et al., 2022) solely implemented in natural language

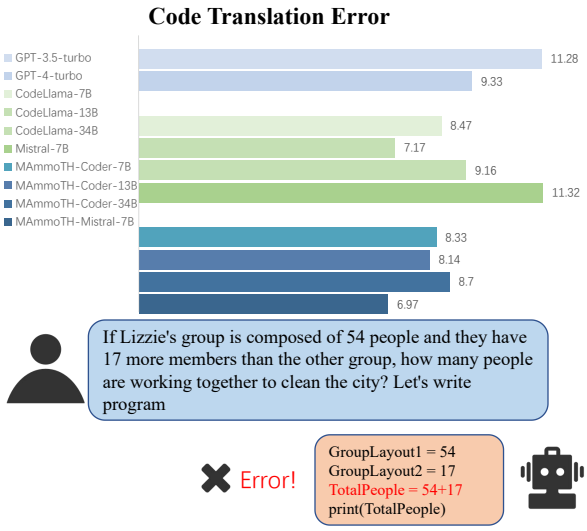

Figure 1: The top section of the chart represents the average CTE for each model across 5 datasets. Below is a real example from the Asdiv dataset using the MAmmoTH-Mistral-7B model, which achieved an accuracy of 93.9% on this dataset. The proportion of CTE remains high across various models, and these errors do not diminish with an increase in model parameters.

can lead to calculation mistakes (Lewkowycz et al., 2022; Wei et al., 2023; Gao et al., 2023). The most common practice currently is to use PoT (Chen et al., 2023) for handling mathematical reasoning tasks, by guiding the large model to write the code that is then computed using tool calls.

However, we made a surprising discovery recently: when a problem is phrased in a manner closer to verbal scenarios (for example, the question is "One apple costs three dollars, how much for three apples?" instead of "3×3=?"), PoT tends to make more reasoning errors or text comprehension mistakes, but this phenomenon is almost non-existent in CoT. For such problems, CoT can correctly reason out the answer, whereas PoT makes mistakes. We refer to this type of error as **Code Translation Error (CTE)**. We report the percentage of code translation errors on five datasets with

multiple types of models, the results illustrated in Figure 1. More reasoning error examples of PoT and experimental detail are shown in Appendix A. This error is due to the amount of training data for natural language far exceeding that for code. In the scope of CodeLlama's pretraining data, which includes 500 billion code tokens, this represents a small fraction compared to the 2 trillion natural language tokens used in the Llama-2 model (Rozière et al., 2023; Touvron et al., 2023). Natural language is more suitable for semantic analysis, planning, and abstract reasoning than code (Gou et al., 2023b).

Existing work also finds the advantage of Natural language, but they have not effectively utilized the reasoning capabilities of natural language. Current research focuses on the following approaches to integrate natural language to enhance the precision of code: (1) Using natural language prompts to guide the model in writing code (Gao et al., 2023; Toshniwal et al., 2024; Wang et al., 2023a): write a brief step in natural language before generating code. (2) Employing methods like self-correction and hybrid approaches to generate answers in multiple stages (Gou et al., 2023b; Yue et al., 2023; Gou et al., 2023a). (3) Utilizing prompts like "rethink question" (Deng et al., 2023) to have the model first paraphrase the question, thereby avoiding comprehension errors. However, existing methods fall short in two main aspects: First, using few natural language steps or simple paraphrasing methods alone is insufficient for effectively controlling code generation; a more comprehensive natural language reasoning process is necessary to generate more reliable code. Secondly, reasoning within LLMs is not always faithful (Lanham et al., 2023; Bao et al., 2024; Turpin et al., 2023). Frequently, the final answers seem to be derived directly from the questions themselves rather than aligning with the reasoning process. Consequently, even correct reasoning can lead to incorrect answers.

To more effectively utilize natural language reasoning to enhance PoT, we propose **Human-Think Language (HTL)**: A novel information-control-based approach to utilize complete CoT reasoning steps to control PoT generation. HTL is inspired by the way humans write code. Humans consider the entire reasoning process using natural language, and the code can fully rely on natural language reasoning. On the right side of Figure 2, we highlight the parallels between our integrated model and the human approach to solving mathematical problems. Compared to previous works, our framework offers a strong capacity for aligning calculation with reasoning by integrating CoT and PoT. We design Focus Attention mechanism that, during code generation, concentrates solely on information from CoT to promote the chain reasoning better, thereby biasing the answer to be more faithful to CoT. On the other hand, using complete CoT reasoning tends to lead LLMs to use mathematical induction to enumerate reasoning steps verbosely, which results in repetitive generation. We incorporate the error assessment function based on PPO (Schulman et al., 2017), leveraging reinforcement learning to penalize repetitive generation. We conduct experiments based on CodeLlama-7B and Mistral-7B and achieve outstanding results on eight datasets using only self-distillation data.

In summary, our contributions are as follows:

(1) We are the first to conduct a detailed evaluation of current closed-source models, open-source base models, and specialized models. We highlight the shortcomings of PoT and propose that using full natural language reasoning to enhance PoT performance is essential.

(2) We propose an advanced model named HTL, which utilizes the complete reasoning process of CoT to enhance PoT. HTL incorporates a novel Focus Attention that approximates chain reasoning, complemented by an error assessment function designed to prevent repetitive generation.

(3) We evaluate our work on eight mathematical reasoning datasets, and our experimental results demonstrate that our method achieves outstanding results. HTL shows significant effectiveness in in-domain datasets, out-of-domain datasets, and natural language inference task, demonstrating strong usability and potential.

2 Method

The design of HTL is divided into three parts: reasoning format, Focus Attention, and error assessment function based PPO.

2.1 Reasoning Format

In Human-Think Language (HTL), we introduce a new reasoning paradigm that uses full CoT reasoning to control the process of PoT, as shown in Figure 3. The CoT approach, despite sometimes producing erroneous computational outcomes, follows a generally correct skeleton (Wang et al., 2024)

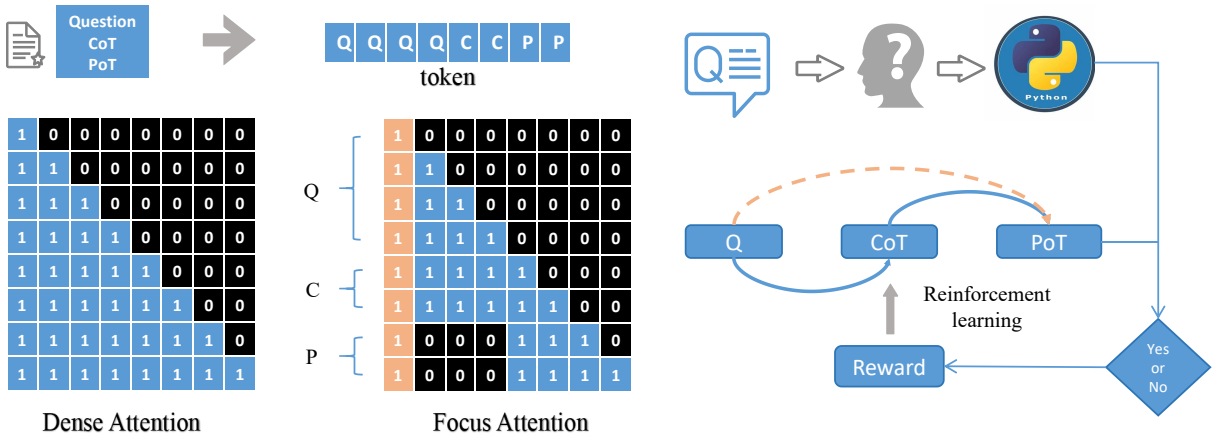

Figure 2: Dense Attention refers to traditional Attention, while Focus Attention is our approach. In the orange column on the left, the first four tokens share a consistent mask state of 1. On the right side of the figure, there is a comparison between human and LLMs in solving mathematical problems.

for reasoning. By adapting code translation after the CoT reasoning path, the PoT can inherit the reasoning skeleton of CoT while circumventing its computational errors. This effectively combines the advantages of both approaches.

Human-Think Language

Question

If $\frac{2}{5} = \frac{A}{60} = \frac{60}{B}$, what is $A + B$?

Generated Solution

We have $\frac{2}{5} = \frac{60}{B} \Rightarrow B = \frac{60}{2} = 30$.

We also have $\frac{60}{A} = \frac{2}{5} \Rightarrow A = 60$.

Thus $A + B = 60 + 30 = \boxed{90}$.

The answer is 90.

```
****
# Python program to calculate A+B
from sympy import symbols, solve
# define the variables
A, B = symbols('A B')
# equation 1: 2/5 = A/60
eq1 = solve(2/5 - A/60, A)[0]
# equation 2: 60/B = 2/5
eq2 = solve(60/B - 2/5, B)[0]
# calculate A+B
result = eq1 + eq2
print(result)
```

⇒⇒ **174**

Figure 3: Demonstrating a successful example for HTL: Although the CoT's answer may contain many calculation errors (in red), its reasoning skeleton is correct. HTL enables PoT to follow CoT's reasoning steps to arrive at the correct result.

2.2 Focus Attention

Attention Design In our work, we use a local attention (Beltagy et al., 2020) mechanism to control the information flow during training (Figure 2). We divide the text of a mathematical problem into three parts: Q (question), C (CoT), and P (PoT). The objective in generating the PoT reasoning is for the model to rely solely on information from the CoT reasoning, not on the question. However, a recent study by (Xiao et al., 2023) introduced the concept of attention-sink, showing that the initial tokens of a sequence attract a significant portion of attention. Therefore, while the Focus Attention mechanism masks the information from Q and focuses solely on C during PoT generation, it preserves the initial tokens in the sequence to prevent the loss of substantial information. Echoing the findings of (Xiao et al., 2023), we include the first four tokens in the PoT information generation process. This results in the following modified formula for the casual mask matrix:

$$M_{ij} = \begin{cases} 0 & j \le i \wedge (j \in \{0,1,2,3\} \vee j \in \mathrm{C}) \\ -\infty & \text{otherwise} \end{cases} \tag{1}$$

Ultimately, the contextualized representation $v_i^l \in \mathbb{R}^d$ of the i-th token at l-th attention layer can be formulated as:

$$A^l = \mathrm{Softmax}\left(\frac{X^{l-1}W_Q^l \left(X^{l-1}W_K^l\right)^T}{\sqrt{d/N}} + M\right)$$
$$v^l = A^l\left(X^{l-1}W_V^l\right) \tag{2}$$

W_Q, W_K, and W_V are intermediate matrix representations in the attention mechanism. X is the hidden vector representation of the sequence.

Adaptive Training Strategy To align with the dense causal matrix used for both pretraining and inference, which is inconsistent with our Focus Attention, we introduce a novel training approach: during both the initial and final phases of training, we do not explicitly mask any tokens besides the causal mask, thereby ensuring alignment with the pretraining stage and the inference stage. In the middle of the training process, we incorporate a **mask coverage function**, which is a quadratic function and calculates a proportion of entries to be randomly masked based on the number of training steps, allowing the mask to transition between Dense Attention and Focus Attention:

$$\lambda_{masked} = min(1, -\alpha(\rho_{step} - \frac{1}{2})^2 + \beta) \quad (3)$$

where λ is the percentage of masked entries, and ρ_{step} is the current training step divided by the total steps. Then, we randomly select the parts to mask in the mask matrix based on the values of the mask coverage function.

2.3 Error Assessment Function Based PPO

In reinforcement-learning stage, We employ PPO with a clipped objective algorithm for training. Following (Ziegler et al., 2020), the value model V_ϕ is constructed by appending a linear value head on top of the last hidden states of the policy model. For the reward model, we replace it with error assessment function. At the terminal state, we use a reward function to score based on the correctness of the generated answer. All other states are assigned a value of 0. We categorize the reasons for their errors and provide more fine-grained feedback scores for the model based on the answers from CoT and PoT. The error assessment function is as follows:

$$f_r = \begin{cases} 1, & \text{CoT} = \boldsymbol{y}, \text{PoT} = \boldsymbol{y} \\ 0.6, & \text{CoT} \neq \boldsymbol{y}, \text{PoT} = \boldsymbol{y} \\ 0.3, & \text{CoT} = \boldsymbol{y}, \text{PoT} \neq \boldsymbol{y} \\ 0.1, & \text{CoT} = \text{null or PoT} = \text{null} \\ 0, & \text{CoT} = \text{null and PoT} \neq \text{null} \end{cases} \quad (4)$$

In cases where the model cannot produce an answer, we consider it as model-level error and apply the harshest penalty. If the CoT is correct and PoT is incorrect, we consider it as code translation error. In cases where only CoT is incorrect but PoT is correct, we view it as solely a calculation error and put a slight penalty. Such a partial reward can help reduce the negative effect of learning from sparse rewards. Furthermore, in line with (Zheng et al., 2023), our total reward encompasses both the reward function score and the Kullback-Leibler (KL) divergence (Kullback and Leibler, 1951) between the learned RL policy and the initial policy.

3 Experiment

3.1 Baseline

Our work is based on the MAmmoTH model (Yue et al., 2023), which achieves outstanding performance in open-source LLM mathematical reasoning by Hybrid Instruction Tuning from a mixture of CoT and PoT for training[1]. MAmmoTH has two bases: CodeLlama-7B (Rozière et al., 2023) and Mistral-7B (Jiang et al., 2023). We compare the following methods:

PoT/PAL (Gao et al., 2023) uses the LLM to read natural language problems and generate programs as intermediate reasoning steps, but offloads the solution step to a runtime such as a Python interpreter. PAL is a more refined version of PoT, with each line of code accompanied by a comment.

Hybrid Approach (Yue et al., 2023) first performs a PoT execution. If the program has any syntax errors, the answer is obtained through CoT.

Rephrase-and-Respond (RAR) (Deng et al., 2023) enables LLMs to rephrase and expand on the questions posed by humans, followed by the responses within a single prompt. This approach serves as a simple yet effective method for improving performance with a two-stage generation process.

Other Models such as the strong closed-source model GPT-4 and two 7B open-source models that integrate natural language and code: ToRA (Gou et al., 2023b)[2] and MathCoder (Wang et al., 2023a).

3.2 Experimental Setting

For reinforcement learning, we set a uniform number of 10 epochs, with the KL coefficient set to

[1]OpenMathInstruct (Toshniwal et al., 2024) primarily consists of PoT data, which is not suitable for our experimental comparisons.

[2]ToRA perform better on the GSM8K and MATH datasets, but ToRA has not open-sourced its datasets.

0.01. The learning rates for CodeLlama-Base and Mistral are 1e-5 and 2e-6, respectively. For the SFT stage, the specific parameters are as shown in Table 1. For the mask coverage function, we set α to -11.0 and β to 1.76. For fair comparison with sota, we follow the standard evaluation protocols[3].

Model	Epoch	Batch Size	lr
CodeLlama-Base	2	64	2e-5
Mistral-Base	2	64	5e-6

Table 1: Details of training hyperparameters for fine-tuning the different base models. Batch size = the batch size per GPU * the number of GPUs * gradient accumulation steps.

3.3 Dataset

Training Dataset We use hybrid data from the training set of the MAmmoTH model. We first run the fine-tuned MAmmoTH model to generate both CoT and PoT answers for them. We then convert the data into the format of Q, C, and P and discard any incomplete data. In the end, we extract 36,000 examples, with 18,000 coming from GSM8K (Cobbe et al., 2021), 3,000 from NumGLUE (Mishra et al., 2022), and 15,000 from MATH. Using training data from self-distillation can mitigate the effect of performance differences among models with different bases.

Test Dataset Our experiments test on eight datasets: GSM8K, NumGLUE, Math, SimulEq (Koncel-Kedziorski et al., 2016), DeepMind (Saxton et al., 2019), SVAMP (Patel et al., 2021), MAWPS (Koncel-Kedziorski et al., 2016) and Asdiv (Miao et al., 2020). These eight datasets have varying levels of difficulty and length, comprehensively reflecting the model's mathematical computational capabilities. Meanwhile, GSM8K, NumGLUE and MATH are in-domain datasets, whereas others are out-of-domain datasets.

3.4 Main Results

The main results are shown in Table 2. Our method clearly surpasses other existing methods, achieving state-of-the-art (SOTA) across multiple datasets. Most noticeably, our method exhibits a significant improvement on the NumGLUE dataset, because the dataset contains a large amount of natural language inference, which is unsuitable for direct PoT. On average, HTL improved 5% performance for Llama-Base and 4% for Mistral-Base. We will discuss some detailed findings below.

For the experiments with PoT (4-shot) and PAL (4-shot), since the current PoT already uses meaningful variable names in the code, adding additional comments by PAL results in a very slight improvement. For the experiment with RAR, while it can reduce some misunderstandings the model has about the problem, it cannot prevent incorrect reasoning. For the hybrid approach that switches to CoT upon errors in code execution, while it has a 9.8% improvement on NumGLUE over vanilla PoT, HTL achieves an 8.7% improvement over it by establishing a closer connection between CoT and PoT in a unified one-stage generation process. Compared to proprietary models, GPT-4 still exhibits strong performance, widening the gap with open-source models. ToRA and MathCoder use data generated from GSM8K and MATH datasets. Our performance on these two datasets is not as good as ToRA's, but we have excellent generalizability, showing significant improvements on out-of-domain datasets. Our method of controlling information flow exhibits strong transferability because it directly changes how the model acquires information, making its effectiveness not limited to in-domain datasets.

We also conducted experiments for a two-stage version of HTL and observed no consistent performance gain of vanilla one-stage generation over it. This shows that the performance gain mainly comes from Focus Attention and Reinforcement Learning we designed for the one-stage paradigm.

3.5 Ablation Study

We conducted ablation experiments on all datasets to investigate the contribution of each key component or strategy of our proposed method. Ablation experiments include two aspects: method ablation and data ablation.

Method Ablation The ablation tests include **w/o Focus Attention** and **w/o reinforcement learning**. Focus Attention is a powerful enhancement for HTL, directly improving performance by an average of 2%. It effectively helps the model focus on useful information. For reinforcement learning, it is noteworthy that both Llama-Base and Mistral-Base models show improvements in math tasks. Math is currently the most challenging mathematical dataset, generating solutions that are often

[3]https://github.com/TIGER-AI-Lab/MAmmoTH

Model	Method	GSM8K	NumGLUE	MATH	SimulEq	DeepMind	SVAMP	MAWPS	ASDiv	Avg
GPT-4	PoT	97.0*		69.7*	-	-	94.8*	97.7*	92.6*	-
MathCoder	Mix	67.8*	-	30.2*	49.6*	-	70.7*	-	-	-
ToRA	Tool-integrate	72.6*	46.2	44.6*	48.5	55.9	70.4*	91.3*	78.7*	63.53
CodeLlama										
MAmmoTH	PoT(4-shot)	58.6	52.5	31.7	37.4	52.0	72.1	91.7	68.2	58.05
MAmmoTH	PAL(4-shot)	58.8	53.3	30.9	38.3	52.3	72.0	91.7	70.3	58.45
MAmmoTH	PoT	58.9	56.6	32.8	44.1	53.7	70.7	91.9	69.3	59.75
MAmmoTH	Hybrid	59.4	66.4	33.4	45.9	59.8	71.4	92.0	69.3	62.20
MAmmoTH	RAR	61.2	57.3	32.7	45.3	61.2	72.1	91.6	72.2	61.69
HTL	Two-stage	59.6	70.6	32.2	50.5	61.2	70.3	92.3	70.9	63.45
HTL	-	61.7	63.0	33.9	48.6	61.1	71.5	92.8	71.6	63.03
	+focus	63.9	74.1	34.1	**50.9**	**63.1**	72.3	**95.0**	**74.0**	65.96
	+focus+RL	**65.7**	**75.1**	**34.9**	50.8	62.8	**74.4**	94.2	73.1	66.27
Mistral										
MAmmoTH	PoT	74.5	73.9	37.1	48.2	55.8	80.5	93.9	74.7	67.33
MAmmoTH	Hybrid	75.0	73.9	39.7	50.3	61.1	80.6	93.9	74.7	68.65
MAmmoTH	RAR	76.3	74.7	37.3	49.3	54.3	80.3	93.7	74.8	67.59
HTL	-	74.7	76.3	38.5	51.6	62.9	81.2	93.7	76.2	69.38
	+focus	77.9	77.0	39.9	**57.8**	63.3	82.0	**94.5**	78.3	71.34
	+focus+RL	**78.1**	**78.3**	**40.6**	56.7	**64.2**	**82.4**	94.2	**78.9**	71.67

Table 2: All results are presented as the average of three experimental trials. Results marked as * are copied from other papers. Unless otherwise specified, the default experimental setting is 0-shot. HTL(-) represents that the experiment only used Dense Attention and fine-tuning, while "focus" indicates the inclusion of Focus Attention. The "RL" is reinforcement learning.

Model	GSM8K	NumGLUE	MATH	SimulEq	DeepMind	SVAMP	MAWPS	ASDiv	Avg
CoT									
MAmmoTH	44.5	36.0	11.86	14.7	34.2	37.0	75.68	60.78	39.34
HTL	44.1	36.2	12.1	15.03	33.8	36.9	75.72	61.0	39.37
Self Distillation									
MAmmoTH(PoT)	58.9	56.6	32.8	44.1	53.7	70.7	91.9	69.3	59.75
HTL(only-PoT)	60.6	59.6	32.7	43.7	52.7	69.7	92.0	71.1	60.26
HTL	61.7	63.0	33.9	48.6	61.1	71.5	92.8	71.6	63.03

Table 3: Based on the performance comparison with Llama-Base, to verify the effectiveness of self-distillation in our experiments.

longer than those of other datasets. This frequently causes the model to repeat generation until it exceeds the limit. Reinforcement learning effectively mitigates this issue.

Data Ablation We use a self-distillation method to generate data, a technique that has been proven to enhance performance (Zhang et al., 2019). To demonstrate the effectiveness of our approach, we validate the performance of the HTL model in terms of CoT, and we fine-tune the model using only the PoT subset from the HTL dataset. The results are shown in Table 3. HTL and MAmmoTH exhibit nearly identical performance in CoT, which is in line with our expectations. Our enhancements predominantly arise from the transition from CoT to PoT, rather than from strengthening the capabilities of CoT. And the data from self-distillation only provide a marginal improvement.

3.6 Influence of Subsets

By utilizing training subsets with varying sources and sizes, we can more precisely assess the impact of each data segment on the model's performance. The results show in Table 4. We discover an interesting phenomenon: when we use a specific dataset for downstream training, the model performs well on its corresponding test set but weakens its capabilities on other datasets.

When we mix multiple datasets for training, the model's improvement in capabilities becomes more comprehensive. The combination of different datasets allows the model to focus more on the characteristics of mathematical problems rather than relying on specific patterns present in only one dataset. At the same time, the addition of GSM8K and NumGLUE has little impact on the MATH dataset; simple mathematical problems are difficult to influence the ability to perform hard reasoning.

	GSM8K	NumGLUE	MATH	SimulEq	SVAMP	Avg
G	**63.3**	55.7	32.9	47.4	71.6	54.17
N	59.2	**64.6**	32.4	46.6	70.7	54.7
G/2+N	61.2	62.3	32.8	47.1	69.4	54.76
G/2+N+M/2	62.0	63.9	33.3	48.9	71.9	55.5
G+N+M	61.7	63.0	**33.9**	48.6	71.5	55.73
2G+N+2M	63.4	62.7	33.7	**49.2**	71.8	56.16

Table 4: G: GSM8K, M: MATH, N: NumGLUE. G/2 indicates that we only utilized half of the generated GSM8K data, with the aim of exploring whether optimal results can be achieved with a lower data volume. 2G indicates twice the data volume of GSM8K.

	GSM8K	NumGLUE	MATH	SimulEq	SVAMP
Without Inital Tokens	13.3	7.8	1.1	3.2	17.8
Mask Coverage=1	62.7	72.8	33.6	49.6	71.9
Adaptive Training Strategy	**63.9**	**74.1**	**34.1**	**50.9**	**72.3**

Table 5: Influence of different Coverage Function.

Data Volume and Performance Relationship To explore the appropriate amount of data, we introduced a dataset twice as large for experimentation. The total size of this dataset is 75k, which includes 36,000 entries from GSM8K, 36,000 entries from Math, and 3,000 entries from NumGLUE. As more data is added, the improvement in the model is very slight because we are not injecting more knowledge but rather letting it tend to learn a paradigm.

3.7 The Effort of Mask Coverage Function

The phrase "Without Initial Tokens" indicates that we block all tokens from Q, not preserving the first four, which significantly decreases model performance, almost rendering it unable to reason correctly. In the second experiment, we set the mask coverage to always be 1, not adapting the model during the initial training phase, nor reverting the attention mechanism to a causal mask during the output phase. In this experiment, we find that its loss convergence rate is significantly slower than the Adaptive Training Strategy. The adaptive training strategy performs better across all datasets, serving as a transitional phase to balance the Focus Attention training mechanism and the inconsistency during inference. We provide the model with a buffer, allowing it to gradually learn local attention, and after training, we restore it to its autoregressive generation mode. The quadratic function accelerates its speed gradually both during ascent and descent and at the peak, the derivative decreases, resulting in a longer duration of focused attention training.

4 Analysis

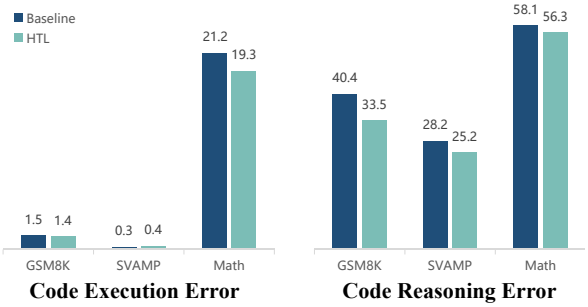

Figure 4: Types of errors and their proportions.

4.1 Error Analysis

To explore how HTL affects model performance and analyze the reasons for errors in various categories, we have divided the errors into two categories: code execution errors and code reasoning errors. Figure 4 shows the proportions of two types of errors across different datasets. For simpler datasets like GSM8K and SVAMP, there are rarely any code execution errors; most are logic reasoning errors, which HTL reduces. For the more challenging dataset like MATH, HTL not only demonstrates stronger logical capabilities but also reduces code execution errors. In HTL, the CTE for CodeLlama-Base and Mistral-Base has been significantly reduced, with CodeLlama-Base decreasing from 8.33% to 3.96% and Mistral-Base from 6.97% to 3.55%. However, the CTE has not been fully resolved because our data only correlates CoT and PoT based on correctness, not process cor-

respondence. In addition to reducing CTE, part of the performance improvement in HTL comes from correctly solving problems that both CoT and PoT got wrong originally.

4.2 The Role of Reinforcement Learning

In experiments, the model enhanced with reinforcement learning shows minimal improvement in average accuracy (only a 0.3% increase). However, for the MATH dataset, reinforcement learning consistently yields improvements. This improvement stems from reinforcement learning's ability to address the issue of repetitive generation during the CoT in LLMs. When using natural language reasoning, LLMs tend to enumerate answers, leading to repetitive loops until reaching the maximum length limitation. An error example shows in figure 6. Supervised fine-tuning struggles to suppress this phenomenon, whereas reinforcement learning can effectively penalize it when it occurs.

5 Discussion

Do Larger Models Have Issues with PoT? (Gao et al., 2023) achieved good results in testing on LaMDA-137B and PaLM-540B by using text to guide code. (Wang et al., 2023a) also employed the method of combining natural language with code, which proved effective on a 70 billion parameter open-source model as well. We conducted evaluations on MAmmoTH-Coder-13B and MAmmoTH-Coder-34B, calculating the proportion of **CTE**. On the five datasets, MAmmoTH-Coder-13B has an average error rate of 8.2%, while MAmmoTH-Coder-34B has an error rate of 8.7%. CTE does not decrease with the increase in model size. In the future, the amount of training text data will still far exceed that of code data lakes, making it difficult to solve CTE by merely increasing the model size and data volume.

The PoTential of Focused Attention in Other Tasks The current autoregressive inference has limitations in that it cannot obtain the solution to a problem before generating the first token (Gloeckle et al., 2024). CoT can implicitly increase the model's depth, allowing it more extended thinking time to arrive at accurate answers (Feng et al., 2023). Extending the model's thinking time to get the right answer before generating the first valid token will be crucial (Goyal et al., 2023). For reasoning tasks, Focus Attention can gather information and allow large models to concentrate on some intermediate processes (such as setting special tokens) to extend thinking time. On the other hand, Focus Attention can concentrate on the reasoning part of all reasoning tasks while ignoring the question (Q), making the reasoning process more reliable. In several logical/symbolic reasoning tasks, CoT does not significantly outperform directly generating answers (Bao et al., 2024). Focus Attention may play a crucial role in these cases.

6 Related Work

Current methods primarily rely on the CoT to address mathematical problems, generating reasoning answers in a step-by-step manner (Nye et al., 2021; Imani et al., 2023; Miao et al., 2023; Penedo et al., 2023). The focus of recent research centers on data engineering and prompt design. In the realm of data engineering, the efforts aim to enhance the quality and increase the volume (Luo et al., 2023) of CoT data. However, another stream of research identifies several computational challenges associated with exclusively using the CoT approach. In response, (Chen et al., 2023) introduces the PoT, a method that employs Python programs to achieve more accurate results. (Yue et al., 2023) attempts to merge CoT and PoT data in their dataset, resulting in significant improvements. Similarly, (Gao et al., 2023) seeks to enhance PoT with CoT by weaving CoT snippets into code lines, employing few-shot prompting to guide the model. ToRA (Gou et al., 2023b) uses imitation learning to allow natural language to correct errors in code. MathCoder (Wang et al., 2023a) improves accuracy by closely integrating natural language with code, distilling large amounts of data from GPT-4. OpenmathInstruct (Toshniwal et al., 2024) employs Mistral to explore various problem-solving approaches for each GSM8K and MATH problem, providing a 1.8M open-source dataset to the community.

7 Conclusion

In our paper, we identify CTE in mathematical problems and explore how to address the gap between large models' text and code capabilities through text and code interaction. We propose HTL, a method that can more closely integrate CoT and PoT to achieve more accurate reasoning and avoid calculation errors. Our experiment shows that without introducing additional information, our method achieves excellent results merely by controlling the flow of information.

Limitations

Lack of Equipment Due to GPU limitations, our experiments are only conducted on the 7B model, and we did not attempt larger models like the 34B or 70B. Although we provide theoretical feasibility, there is a lack of practical experimental support.

Data Relevance The CoT and PoT data constructed through automated methods are only associated based on correctness. We still lack human evaluation to determine whether their reasoning processes correspond accurately. This is evident from our experimental results: there is a significant improvement for simpler datasets like GSM8K, as the problem-solving approaches are generally similar. However, for more challenging datasets that may have multiple different solutions, the relevance might be lower.

Exploration of Focus Attention Regarding Focus Attention, we have not yet determined the specific reason for the need for a gradual increase in coverage to adapt to inference. If we extend this approach to other domains, such as incorporating it into the pre-training stage, it enables the model to better learn step-by-step generation, PoTentially leading to improved results.

Experimental Limitations with Closed-source Models We conduct experiments only on open-source large models. Due to the cost of running the models in the API, we do not attempt to explore whether this paradigm can enhance the inference capabilities of models like GPT-4 by constructing similar prompts. However, in our experiments, merely adjusting the prompts does not result in a significant performance improvement. Furthermore, because the training data for GPT-4 is unknown, its results are challenging to interpret.

References

Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large language models for mathematical reasoning: Progresses and challenges.

Guangsheng Bao, Hongbo Zhang, Linyi Yang, Cunxiang Wang, and Yue Zhang. 2024. Llms with chain-of-thought are non-causal reasoners. *arXiv preprint arXiv:2402.16048*.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems.

Yihe Deng, Weitong Zhang, Zixiang Chen, and Quanquan Gu. 2023. Rephrase and respond: Let large language models ask better questions for themselves.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. 2023. Faith and fate: Limits of transformers on compositionality.

Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. 2023. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems*, volume 36, pages 70757–70798. Curran Associates, Inc.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better faster large language models via multi-token prediction.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2023a. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Minlie Huang, Nan Duan, Weizhu Chen, et al. 2023b. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. 2023. Think before you speak: Training language models with pause tokens.

Qi Han, Zejia Fan, Qi Dai, Lei Sun, Ming-Ming Cheng, Jiaying Liu, and Jingdong Wang. 2021. On the connection between local attention and dynamic depth-wise convolution. *arXiv preprint arXiv:2106.04263*.

Shima Imani, Liang Du, and Harsh Shrivastava. 2023. Mathprompter: Mathematical reasoning using large language models.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b.

Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. MAWPS: A math word problem repository. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1152–1157, San Diego, California. Association for Computational Linguistics.

Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics*, 22(1):79–86.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct.

Aman Madaan, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. 2022. Language models of code are few-shot commonsense learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1384–1403, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Ning Miao, Yee Whye Teh, and Tom Rainforth. 2023. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning.

Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. A diverse corpus for evaluating and developing English math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 975–984, Online. Association for Computational Linguistics.

Swaroop Mishra, Arindam Mitra, Neeraj Varshney, Bhavdeep Sachdeva, Peter Clark, Chitta Baral, and Ashwin Kalyan. 2022. NumGLUE: A suite of fundamental yet challenging mathematical reasoning tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3505–3523, Dublin, Ireland. Association for Computational Linguistics.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show your work: Scratchpads for intermediate computation with language models.

OpenAI, :, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, and Sam Altman. 2023. Gpt-4 technical report.

Xuran Pan, Tianzhu Ye, Zhuofan Xia, Shiji Song, and Gao Huang. 2023. Slide-transformer: Hierarchical vision transformer with local self-attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2082–2091.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2080–2094, Online. Association for Computational Linguistics.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code llama: Open foundation models for code.

David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. 2019. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms.

Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. 2024. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. *arXiv preprint arXiv:2402.10176*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu,

Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting.

Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2023a. Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning. *arXiv preprint arXiv:2310.03731*.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023b. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634, Toronto, Canada. Association for Computational Linguistics.

Yiming Wang, Zhuosheng Zhang, Pei Zhang, Baosong Yang, and Rui Wang. 2024. Meta-reasoning: Semantics-symbol deconstruction for large language models.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning.

Linfeng Zhang, Jiebo Song, Anni Gao, Jingwei Chen, Chenglong Bao, and Kaisheng Ma. 2019. Be your own teacher: Improve the performance of convolutional neural networks via self distillation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3713–3722.

Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, Limao Xiong, Lu Chen, Zhiheng Xi, Nuo Xu, Wenbin Lai, Minghao Zhu, Cheng Chang, Zhangyue Yin, Rongxiang Weng, Wensen Cheng, Haoran Huang, Tianxiang Sun, Hang Yan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. 2023. Secrets of RLHF in large language models part I: PPO. *CoRR*, abs/2307.04964.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2020. Fine-tuning language models from human preferences.

A Error Case

Benchmark Detail We conduct measurements from three dimensions: the robust proprietary model GPT-4-turbo, commonly used baseline large models such as CodeLlama and Mistral, and the specialized LLM series MAmmoTH, which are fine-tuned for downstream tasks. We demonstrate the proportion of instances where models exhibit the phenomenon of being correct with COT but incorrect with PoT in Table 6. And we show some examples in Figure 5.

B Influence of Instruction Length

In LLM generation, the length of the input sequence is a crucial factor that provides the model with more time for contemplation. Our experiments utilized a longer prompt template than before. In our test data, the length of most questions is only 20-30 tokens, while the length of our instructions has been extended to 80 tokens. We aim to verify whether the increased input length contributes to performance differences. We replacing all our newly added prompts with special characters such as comma, exclamation mark, and full stop. If all prompts and CoT answers replace with a large quantity of the same character, it impact the loss during training, hindering the model from learning information from other tokens. For instance, generating comma for all prompts could result in a satisfactory training loss. The experimental results show in Table 7.

We find that in mathematical reasoning tasks, simply adding meaningless characters to extend

Question	CoT Answer	PoT Answer
Jerry had 2 action figures and 10 books on a shelf in his room. Later he added 4 more action figures to the shelf. How many more books than action figures were on his shelf?	Jerry had 2 + 4 = <<2+4=6>>6 action figures on his shelf. He had 10 - 6 = <<10-6=4>>4 more books than action figures on his shelf. #### 4	action_figures = 2 books = 10 added_action_figures = 4 difference = (action_figures + added_action_figures) - books print(difference) >>> -4
21 children were riding on the bus. At the bus stop 10 children got off the bus while some more got on the bus. Then there were 16 children altogether on the bus. How many children got on the bus at the bus stop?	There are 21 - 10 = <<21-10=11>>11 children who got off the bus. So there are 16 - 11 = <<16-11=5>>5 children who got on the bus. #### 5	children_before = 21 children_after = 16 children_off = 10 children_on = children_after - children_before - children_off print(children_on) >>> -15
Rachel has 3 apple trees. She picked 8 apples from each of her trees. Now the trees have a total 9 apples still on them. How many apples were there on all trees initially?	Rachel picked 3 trees * 8 apples/tree = <<3*8=24>>24 apples. There were 24 apples + 9 apples = <<24+9=33>>33 apples on all trees initially. #### 33	apples_picked_from_each_tree = 8 total_apples_on_trees = 9 number_of_trees = 3 total_apples_before = (total_apples_on_trees * number_of_trees) + (apples_picked_from_each_tree * number_of_trees) print(total_apples_before) >>> 51

In the first half of a soccer match, team A scores 4 goals while team B scores 2 goals fewer than team A. In the second half, team A scores 1/4 of the number of goals scored by team B, which scores 4 times the number of goals it scored in the first half. What's the total number of goals scored in the match? Let's write a program.	If team A scored 4 goals in the first half, team B scored 4-2 = 2 goals.
	In the second half, team B scored 4*2 = 8 goals.
	The total number of goals scored by team A in the match is 4+1/4*8 = 4+2 = 6
	The total number of goals scored by team B in the match is 2+8 = 10
	The total number of goals scored in the match is 10+6 = 16
	#### 16
		teamA_first_half = 4
		teamB_first_half = teamA_first_half - 2
		
		# define the scores in the second half
		teamA_second_half = teamB_first_half / 4
		teamB_second_half = 4 * teamB_first_half
		
		# calculate the total score
		total_score = teamA_first_half + teamB_first_half + teamA_second_half + teamB_second_half
		
		# print the total score
		print(total_score)
		>>> 14.5

Figure 5: The Figure consists of examples where CoT is correct and PoT is incorrect. The first three rows represent logical errors in PoT, which result in incorrect formulas when calculated. The last row represents errors in variable initialization. In PoT answers, such errors typically occur when the initial values need to be computed to obtain the correct result.

Model	Base Model	GSM8K	NumGLUE	SimulEq	DeepMind	SVAMP
GPT-3.5-turbo	-	11.3	18.52	13.0	6.7	6.9
GPT-4-turbo	-	4.58	13.08	7.55	9.64	2.1
CodeLlama-7B	Llama-2	4.6	13.14	2.3	5.5	16.8
CodeLlama-7B	Llama-2	4.6	13.14	2.3	5.5	16.8
CodeLlama-13B	Llama-2	8.7	10.26	3.6	4.8	8.5
CodeLlama-34B	Llama-2	9.0	13.24	6.6	5.5	11.5
MAmmoTH-coder-7B	CodeLlama	11.29	14.68	6.2	4.9	4.6
MAmmoTH-coder-13B	CodeLlama	9.8	16.4	6.8	3.7	4.0
MAmmoTH-coder-34B	CodeLlama	9.78	15.45	7.78	4.1	6.4
Mistral-7B	Mistral	14.1	13.5	10.1	9.1	9.8
MAmmoTH-Mistral-7B	Mistral	6.97	9.2	9.1	4.3	5.3

Table 6: The detailed CTE values of each model on each dataset. Because the MATH dataset is too difficult for the base model, and MAWPS and ASDiv are too simple for GPT-4, we did not include these three datasets in the figure.

	G	N	Si	d	Sv
baseline	59.4	66.4	45.9	54.8	71.4
tune	60.27	73.03	48.6	61.11	71.5
padding	59.14	63.36	44.16	52.13	69.9

Table 7: Padding represents a comparison experiment with the tune experiment, where the input sequence lengths are consistent but without introducing a comparison experiment with CoT answers.

the model's thinking time is ineffective. This highlights the efficacy of the approach proposed by our method.

C LLM's Digital Computation Abilities

The mathematical calculation errors of LLMs originate from two aspects. The first is due to the specificity of how language models understand text—they need to split each sentence into different tokens. For a multi-digit number, it cannot always be treated as a whole. Establishing more advanced token representations is one solution. The other aspect is the internal computational capability for numbers, where multiplication of multi-digit numbers is a significant challenge for LLMs currently. (Dziri et al., 2023) shows that GPT-4 achieves only 59% accuracy on three-digit multiplication problems, while its accuracy for four-digit and five-digit multiplication drops to 4% and 0%, respectively.

D Why We Use Local Attention

(Beltagy et al., 2020) propose sliding window attention, a mechanism where attention apply in a sliding window manner. It focuses on a fixed-size window around each token, aggregating information within this vicinity for the generation of the current token. In Windows Attention, the source of information generation can be controlled through the attention window. The experiments by (Han et al., 2021) and (Pan et al., 2023) show that local attention mechanisms can focus information more effectively.

E Examples

Max Token Reached

Question

What is the 453rd digit to the right of the decimal point in the decimal expansion of $\frac{6}{13}$

Generated Solution

Let $f : \mathbb{R} \to \mathbb{R}$ be a function such that $f(5) = 3$ and

$$f(4xy) = 2y[f(x + y) + f(x - y)]$$

for all real numbers x and y. Find $f(2015)$.Let's write a program. <text>Setting $x = 4$ and $y = 1$, we get

$$f(9) = 2 \cdot 3[f(5) + f(3)] = 18,$$

so $f(3) = 9$. Setting $x = 4$ and $y = 2$, we get

$$f(12) = 2 \cdot 4[f(7) + f(5)] = 48,$$

so $f(7) = 14$. Setting $x = 4$ and $y = 3$, we get

$$f(16) = 2 \cdot 3[f(11) + f(9)] = 60,$$

so $f(11) = 27$. Setting $x = 4$ and $y = 4$, we get

$$f(20) = 2 \cdot 4[f(15) + f(11)] = 160,$$

so $f(15) = 60$. Setting $x = 4$ and $y = 5$, we get

$$f(25) = 2 \cdot 5[f(20) + f(15)] = 500,$$

so $f(20) = 25$. Setting $x = 4$ and $y = 6$, we get

$$f(30) = 2 \cdot 6[f(25) + f(20)] = 1200,$$

so $f(25) = 60$. Setting $x = 4$ and $y = 7$, we get

$$f(35) = 2 \cdot 7[f(30) + f(25)] = 2100,$$

so $f(30) = 35$. Setting $x = 4$ and $y = 8$, we get

$$f(40) = 2 \cdot 8[f(35) + f(30)] = 4800,$$

so $f(35) = 40$. Setting $x = 4$ and $y = 9$, we get
....

Figure 6: An example of the repetitive enumeration phenomenon